

Python Power-Ups Placement Handbook

Chapter 1 (Part 1): PY1.1 - Comprehensions, Lambda & LEGB

Introduction

Python comprehensions provide a concise way to build collections. This chapter teaches not only the syntax but why comprehensions exist, how Python evaluates them, interview tips, pitfalls and real-world applications.

Python comprehensions provide a concise way to build collections. This chapter teaches not only the syntax but why comprehensions exist, how Python evaluates them, interview tips, pitfalls and real-world applications.

Python comprehensions provide a concise way to build collections. This chapter teaches not only the syntax but why comprehensions exist, how Python evaluates them, interview tips, pitfalls and real-world applications.

Why Comprehensions?

Before comprehensions programmers wrote loops with `append()`. Comprehensions make code shorter, clearer and often faster.

Before comprehensions programmers wrote loops with `append()`. Comprehensions make code shorter, clearer and often faster.

Before comprehensions programmers wrote loops with `append()`. Comprehensions make code shorter, clearer and often faster.

List Comprehensions

Syntax: `[expression for item in iterable if condition]`. Explain expression, iterable and optional condition. Example: `squares=[x*x for x in range(6)]`.

Syntax: `[expression for item in iterable if condition]`. Explain expression, iterable and optional condition. Example: `squares=[x*x for x in range(6)]`.

Syntax: `[expression for item in iterable if condition]`. Explain expression, iterable and optional condition. Example: `squares=[x*x for x in range(6)]`.

Dry Run

For `x=0..5`, Python evaluates `x*x` and appends each result to a new list.

For `x=0..5`, Python evaluates `x*x` and appends each result to a new list.

For `x=0..5`, Python evaluates `x*x` and appends each result to a new list.

Filtering

Example: `evens=[x for x in range(10) if x%2==0]`. The condition is checked before storing.

Example: `evens=[x for x in range(10) if x%2==0]`. The condition is checked before storing.

Example: `evens=[x for x in range(10) if x%2==0]`. The condition is checked before storing.

Nested Comprehensions

Matrix flattening and nested loops. Example: `[n for row in matrix for n in row]`.

Matrix flattening and nested loops. Example: [n for row in matrix for n in row].

Matrix flattening and nested loops. Example: [n for row in matrix for n in row].

Dictionary Comprehensions

Syntax: {k:v for ...}. Example: squares={x:x*x for x in range(5)}.

Syntax: {k:v for ...}. Example: squares={x:x*x for x in range(5)}.

Syntax: {k:v for ...}. Example: squares={x:x*x for x in range(5)}.

Set Comprehensions

Syntax: {expr for x in data}. Useful for uniqueness.

Syntax: {expr for x in data}. Useful for uniqueness.

Syntax: {expr for x in data}. Useful for uniqueness.

Generator Expressions

(x*x for x in nums) creates a lazy generator instead of a list.

(x*x for x in nums) creates a lazy generator instead of a list.

(x*x for x in nums) creates a lazy generator instead of a list.

Lambda Functions

Anonymous one-line functions. Common with map(), filter(), sorted(key=lambda x:x[1]).

Anonymous one-line functions. Common with map(), filter(), sorted(key=lambda x:x[1]).

Anonymous one-line functions. Common with map(), filter(), sorted(key=lambda x:x[1]).

LEGB Rule

Local → Enclosing → Global → Built-in. Python searches variables in this order.

Local → Enclosing → Global → Built-in. Python searches variables in this order.

Local → Enclosing → Global → Built-in. Python searches variables in this order.

Common Mistakes

Confusing generator expressions with tuples, overusing lambda, using comprehensions for complex logic.

Confusing generator expressions with tuples, overusing lambda, using comprehensions for complex logic.

Confusing generator expressions with tuples, overusing lambda, using comprehensions for complex logic.

Interview Questions

Difference between list comprehension and generator expression? When should lambda be avoided? Explain LEGB with an example.

Difference between list comprehension and generator expression? When should lambda be avoided? Explain LEGB with an example.

Difference between list comprehension and generator expression? When should lambda be avoided? Explain LEGB with an example.

Revision

Remember: readability first. Use comprehensions for collection creation, lambda for small expressions, and understand LEGB for scope debugging.

Remember: readability first. Use comprehensions for collection creation, lambda for small expressions, and understand LEGB for scope debugging.

Remember: readability first. Use comprehensions for collection creation, lambda for small expressions, and understand LEGB for scope debugging.

Practice Exercises

1. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
2. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
3. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
4. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
5. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
6. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
7. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
8. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
9. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
10. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
11. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
12. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
13. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
14. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.
15. Write a comprehension/lambda solution and then rewrite it using a normal loop. Explain the time complexity and readability trade-offs.